课程笔记：强化学习基础（Policy Gradients）

Bowen-mantou（整理）

2026-10-23

视频作者/频道：B 站「常青藤名校课」· CMU 11-768 AI Agents（英文原声字幕版）
发布日期：2026-09-09
视频时长：77:46
视频链接：https://www.bilibili.com/video/BV1a6Yx62EH4?p=23
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 引言：从 SFT 到策略梯度 2

1.1 运行例子：Number Search . . . 2
1.2 本讲路线图 . . . 2
1.3 本章小结 . . . 2

2 SFT 为什么不够：三个问题 2

2.1 任务失配（task mismatch） . . . 2
2.2 数据失配（data mismatch） . . . 3
2.3 暴露偏差（exposure bias） . . . 3
2.4 本章小结 . . . 3

3 把交互形式化：轨迹与期望奖励 3

3.1 Agent–环境循环 . . . 3
3.2 历史、轨迹与轨迹概率 . . . 4
3.3 目标函数：期望奖励 . . . 4
3.4 本章小结 . . . 4

4 最简的 RL：专家迭代与 REST 4

4.1 REST 的两步循环 . . . 5
4.2 好轨迹不够怎么办 . . . 5
4.3 本章小结 . . . 5

5 策略梯度与 REINFORCE 5

5.1 为什么不能直接反向传播 . . . 5
5.2 对数导数技巧（log-derivative trick） . . . 5
5.3 实现：就是「带 mask、带奖励缩放的 SFT」 . . . 6
5.4 与 REST 的关系 . . . 6
5.5 本章小结 . . . 6

6 基线（baseline）与优势（advantage）：奖励应该是相对的 6

6.1 REINFORCE 的问题：失败轨迹不产生梯度 . . . 6
6.2 优势函数 . . . 7
6.3 减去 baseline 不改变梯度期望 . . . 7
6.4 二动作例子：方差如何被压低 . . . 7
6.5 本章小结 . . . 8

7 GRPO 与 Dr. GRPO：用组均值当基线 8

7.1 GRPO 的核心：同题多采样，组均值作 baseline . . . 8
7.2 Dr. GRPO：两处修正 . . . 9
7.3 全零/全一奖励：没有组内方差就没有信号 . . . 9
7.4 GRPO 与 PPO 的关系 . . . 9
7.5 本章小结 . . . 9

8 三种方法的总账：都在「重加权生成的动作」 **9**

9 总结与延伸 **10**

1 引言：从 SFT 到策略梯度

这是三讲强化学习的第一讲：上一讲学的是用**示范**做监督微调，这一讲开始让模型在环境里**自己试**，用奖励来调整权重。后面两讲会分别讲高级 RL 算法（off-policy、稳定性）与 RL 系统（框架、超参）；本讲的目标是把「策略梯度」这条最基础、也最优雅的线索讲清楚。

1.1 运行例子：Number Search

课堂用一个极简的猜数游戏作为贯穿全讲的可视化例子：

- 环境里藏着一个 1 到 16 之间的整数，agent 不知道；
- agent 最多猜 4 次；每次猜完，环境反馈「太高/太低/正确」；
- 猜对立刻结束、奖励为 1；4 次都没猜对则超时、奖励为 0。

这个游戏的最优首猜是 8（二分空间，像 Wordle 的最优首猜一样按信息论缩小搜索空间）；首猜 4 虽然次优，但只要正确利用反馈也能成功；而首猜 16、或从 1 开始顺序往上猜，则很容易在 4 次内超时。

课堂也定义了奖励：奖励是一个标量，用来衡量一条轨迹的好坏。在 agent 场景里，它可以是「代码是否通过 SWE-bench 的全部单元测试」（通过 1、不通过 0），也可以是「通过了多少比例的测试」（非二值）。今天的方法对二值与非二值奖励都适用；而当前一大类训练用到的是 **RLVR**（reinforcement learning with verified rewards），奖励往往是 0/1。

1.2 本讲路线图

本讲的逻辑是一条直线：先说明 SFT 为什么不够（三个问题）→ 把 agent 与环境的交互形式化成轨迹与期望奖励 → 给出最简单的「采样—筛选—SFT」方法 REST → 推导策略梯度与 REINFORCE → 引入 baseline 与 advantage（为什么奖励要「相对」）→ 最后用 GRPO 把 baseline 换成「同题多采样的组均值」，并介绍 Dr. GRPO 的两处修正。

1.3 本章小结

Number Search 把 RL 的核心要素压到最小：策略、动作、观测、奖励、轨迹。奖励是标量，可以是二值的（RLVR 常见）或连续比例；本讲的目标是最大化期望奖励，而不是模仿某条示范。

2 SFT 为什么不够：三个问题

课堂用同一个 Number Search 例子，把 SFT 的局限归纳成三个问题——它们也正是 RL 要解决的。

2.1 任务失配（task mismatch）

SFT 最大化的是「示范动作」在每个时间点的概率；我们真正想要的却是「任务成功」的概率。两者接近但不同，原因有二：

1. **同一任务有多条成功路径**。首猜 8 最优，但首猜 4 也能成功；拿到「higher」反馈后，8 以上的数字都是合理动作，而示范只会展示其中一条路径。

2. **有些动作明显更差，但 SFT 一视同仁**。例如反馈是「higher」时，正确答案一定在 8 以上，此时猜一个比 8 更小的数字是坏动作；但 SFT 只对「被示范的那个动作」算损失，未示范的动作无论好坏都没有区别信号。

2.2 数据失配（data mismatch）

SFT 只能从「示范」学习；我们希望也能从**次优甚至失败**的轨迹里学到东西。例如一条超时失败的轨迹：我们不想最大化它的动作概率，但希望有一种方法能让模型**避免**这类轨迹。RL 正是这样一个工具：它为失败的轨迹提供负向信号。

2.3 暴露偏差（exposure bias）

SFT 训练时，模型在**固定的历史**上预测下一个动作；推理时，历史是模型自己一步步生成的。如果模型在推理时犯了一个训练中没见过的错误，它就进入了一个分布外（OOD）的历史，此后的行为可能完全不校准。课堂的例子是：如果训练数据里「反馈 higher 之后一定猜更大的数」，那么模型在推理时一旦误猜了一个更小的数，它可能就彻底乱掉。

一个理解 RL 的角度是：**它让模型生成自己的训练数据**（on-policy），并学会从自己的错误中恢复。这样监督信号与模型当前能产生的行为更匹配；代价是采样效率更低（需要真的到环境里试），这也是后面 off-policy 方法要解决的问题。

三个问题要能分别讲清

任务失配 = 目标不一致（模仿动作 vs 最大化奖励）；数据失配 = 只学好的、不学坏的；暴露偏差 = 训练历史来自别人、推理历史来自自己。三个问题分别对应 RL 的三个卖点，面试/作业里要能对号入座。

2.4 本章小结

SFT 优化「复现示范动作的概率」，而我们想要「任务成功的概率」：多路径与坏动作让两者不等价；它无法利用失败轨迹的负向信号；它还会在推理时被自己产生的 OOD 历史带偏。RL 通过「自己采样、按奖励调整」同时缓解这三点。

3 把交互形式化：轨迹与期望奖励

3.1 Agent–环境循环

agent 由策略 π_θ 表示，θ 是语言模型参数。每个时间步，agent 根据历史产生动作（对语言模型而言就是采样 token / 工具调用），动作进入环境，环境返回观测与奖励。很多设定只在任务结束时给奖励（例如跑完单元测试），但本讲方法也适用于中间奖励。

课堂举了三种把生成任务套进这个框架的方式：纯文本生成（动作是下一个 token，奖励来自 judge 模型，例如 RLHF）；web agent（观测是当前页面，动作是点击/输入/滚动，奖励可以是已完成子任务数）；Number Search（隐藏数字是任务实例，观测是 higher/lower/correct，动作是下一个猜测）。

3.2 历史、轨迹与轨迹概率

记 h_t 为到第 t 步为止的**历史**（所有动作与观测的交错序列），τ 为一条**轨迹**（历史加上中间与最终奖励）。策略给出条件分布 $\pi_\theta(a_t \mid h_t)$：给定历史，下一个动作的概率。

轨迹的概率可以分解成「策略」与「环境」两部分的乘积：

$$p_\theta(\tau \mid z) = \prod_t \underbrace{\pi_\theta(a_t \mid h_t)}_{\text{agent 的动作概率}} \cdot \underbrace{p_{\text{env}}(o_t, r_t \mid h_t, a_t, z)}_{\text{环境给观测与奖励的概率}},$$

其中 z 是任务实例，o_t 是第 t 步观测，r_t 是第 t 步奖励，h_t 包含此前所有动作与观测。这个分解只是链式法则；关键在于 **RL 算法不需要知道环境那一项的具体形式**——我们通过「在环境里实际执行动作」来采样它。这也是 RL 慢的原因：它必须真的试、真的看结果，才能得到关于参数的间接反馈。

为什么是 POMDP

经典 RL 常假设马尔可夫性：下一步状态只依赖当前状态与动作。这里我们用的是**部分可观测**设定（POMDP）：环境有隐藏状态（例如隐藏数字、数据库内容），agent 只能看到观测。因此下一个观测的概率必须依赖**全部历史**而不仅是当前状态；两种写法等价，用历史写法在语言模型场景里更顺手。

3.3 目标函数：期望奖励

训练目标是最大化期望奖励

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\cdot \mid z)}\left[R(\tau)\right], \qquad R(\tau) = \sum_t r_t,$$

其中 $R(\tau)$ 是整条轨迹的回报（各步奖励之和）。课堂用一个极简可视化说明「期望」的含义：假设策略只能产生 4 条轨迹，概率分别是 0.3、0.2、0.25、0.25，其中前两条奖励为 1（蓝色）、后两条奖励为 0（红色），则期望奖励是

$$J(\theta) = 0.3 \times 1 + 0.2 \times 1 + 0.25 \times 0 + 0.25 \times 0 = 0.50.$$

训练的目标就是把这部分概率质量从红色（失败）轨迹搬到蓝色（成功）轨迹上。

3.4 本章小结

RL 把交互写成「历史—动作—观测—奖励」的轨迹，轨迹概率 = 策略项 × 环境项，目标是最大化期望回报 $J(\theta)$。环境项未知也无妨：采样即可；代价是需要大量交互，训练变慢。

4 最简的 RL：专家迭代与 REST

在进入策略梯度之前，课堂先给了一个「几乎不用改 SFT 就能做 RL」的桥梁：**专家迭代（expert iteration）**，其中一种具体形式叫 **REST**（reinforced self-training）。

4.1 REST 的两步循环

REST 交替做两件事：

1. **Grow（采样）**：用当前策略采样一批轨迹，计算每条轨迹的奖励；
2. **Improve（筛选 + SFT）**：按奖励过滤——二值奖励就只保留奖励为 1 的轨迹——把保留下来的轨迹加入训练集，对模型做 SFT；
3. 重复：随着策略变强，再采样新轨迹加入数据池，在**不断增长的数据集**上训练。

“REST” 里的 “EM” 指它与期望最大化（EM）算法的形式相似：一边「用当前模型采数据」，一边「让模型更像被选中的数据」。

REST 的损失就是筛选后轨迹上的 SFT 损失：

$$\mathcal{L}_{\text{REST}} = -\sum_{\tau:\, R(\tau)=1} \sum_{t} \log \pi_\theta(a_t \mid h_t),$$

其中外层求和只遍历「奖励为 1 的轨迹」τ，内层求和遍历轨迹中的动作位置 t。它与自蒸馏（self-distillation）的区别只在于：REST 用**奖励**做筛选，而自蒸馏可以不过滤、直接在全部自产轨迹上训练。

4.2 好轨迹不够怎么办

REST 与 RL 都有一个前提：模型得能采样到至少一部分高奖励轨迹。如果模型完全做不对，就没有信号。课堂给了几条对策：先用示范做**冷启动 SFT**（R1 等推理模型的常见做法）；用**课程**先从模型能解出的题目开始；设计**中间奖励**（例如先奖励「探索代码仓库」这类有用的中间步骤），再逐步解锁整个任务。

4.3 本章小结

REST 把 RL 压成「采样—按奖励过滤—SFT」两步循环，复用全部 SFT 工程，是二值奖励下简单有效的基线。它的局限是依赖模型能采样到成功轨迹，需要冷启动、课程或中间奖励来兜底。

5 策略梯度与 REINFORCE

5.1 为什么不能直接反向传播

要对 $J(\theta)$ 求梯度，不能直接用链式法则，原因有两个：第一，动作是从策略里**采样**出来的离散选择，参数改变的是动作的概率分布，采样结果对参数的导数几乎处处为零；第二，动作要在**黑盒环境**里执行，环境几乎总是不可微的。梯度必须通过「采样」这条间接路径来估计。

5.2 对数导数技巧（log-derivative trick）

从 $J(\theta) = \sum_\tau p_\theta(\tau) R(\tau)$ 出发，把梯度移进求和：

$$\nabla_\theta J(\theta) = \sum_\tau \nabla_\theta p_\theta(\tau)\, R(\tau) = \sum_\tau p_\theta(\tau)\, R(\tau)\, \nabla_\theta \log p_\theta(\tau),$$

其中最后一步用了恒等式 $\nabla_\theta p_\theta = p_\theta \nabla_\theta \log p_\theta$（即 $\nabla \log p = \nabla p / p$）。这一步把「对概率求导」变成「对对数概率求导」，后者正是 SFT 里已经在算的东西。

而轨迹概率的对数可以拆成动作项之和：环境项与参数无关，求导后消失，于是

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[R(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t \mid h_t) \right].$$

REINFORCE 估计量

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} R(\tau_i) \sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \mid h_{i,t})$$

用采样平均替代期望：先跑 N 条轨迹、拿到各自奖励，再把「奖励 × 对数概率梯度」加起来。奖励只放大或缩小梯度，不改变方向。

5.3 实现：就是「带 mask、带奖励缩放的 SFT」

实际代码里，策略梯度几乎等于 SFT 的损失再乘一个奖励：对模型生成的 token（蓝框）算对数概率，非模型生成的 token（灰框，prompt/环境/脚手架）用 mask 排除，然后把整条轨迹的对数概率之和乘以这条轨迹的奖励 $R(\tau)$：

$$\mathcal{L}_{\mathrm{PG}} = -R(\tau) \sum_t \log \pi_\theta(a_t \mid h_t).$$

对一个动作由多个 token 组成的情况（例如一次工具调用），把这些 token 的对数概率相加即可。

训练循环是：**采样任务实例** → 生成轨迹并**评估奖励** → 计算上述损失 → 反向传播更新。原版 REINFORCE 每个实例只用一条轨迹，因此 batch size 等于任务实例数；GRPO 会为每个实例生成一组多条轨迹，这将在第 7 节展开。

5.4 与 REST 的关系

在二值奖励下，REINFORCE 与 REST 十分相似：两者都是「用奖励缩放对数概率」。差别在于在线程度——REINFORCE 完全在线（采样后立刻更新），REST 则先攒出一个大数据集、再做多个 epoch 的训练。直觉是同一个，工程取舍不同。

5.5 本章小结

策略梯度用对数导数技巧把「不可微的采样 + 黑盒环境」变回可反向传播的形式：$\nabla J = \mathbb{E}[R(\tau) \sum_t \nabla \log \pi_\theta]$。实现上就是「mask 生成 token + 奖励缩放 + SFT 损失」；这也是为什么语言模型上的 RL 实现看起来异常简单。

6 基线（baseline）与优势（advantage）：奖励应该是相对的

6.1 REINFORCE 的问题：失败轨迹不产生梯度

看一条成功轨迹和一条失败轨迹：成功轨迹的所有动作都被奖励 1 放大；失败轨迹的奖励是 0，于是它的所有动作**根本没有梯度**。但我们恰恰想从失败里学习「不要这么做」。类似地，如果某个题

太简单（95% 成功率），偶尔一次的愚蠢错误代价很高，应该被更重地惩罚——这需要把奖励与「预期表现」比较，而不是只看绝对数值。

6.2 优势函数

定义优势（advantage）：在历史 h 下采取动作 a 比「按策略的平均水平」好多少：

$$A^\pi(h,a) = Q^\pi(h,a) - V^\pi(h),$$

其中 $Q^\pi(h,a)$ 是在 h 下采取 a 之后的期望回报，$V^\pi(h)$ 是在 h 下按策略继续的期望回报（基线）。两者都未知时，可以用同一策略的多次采样来估计。

课堂用猜数例子解释：假设已经猜过 8（higher）、12（lower），所以答案在 9–12 之间（9、10、11 三个候选，还有两次机会）。此时猜 11 是不好的：万一不是 11，只剩两个候选却只有一次机会；猜 10 则很好：无论对错都还能二分。于是 $\hat{A}(\text{猜10}) > \hat{A}(\text{猜11})$，优势函数把这种「相对好坏」量化出来。

6.3 减去 baseline 不改变梯度期望

把 REINFORCE 的奖励换成一个「奖励 - 基线」的形式：

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau \left[\sum_t \big(R(\tau) - b(h_t)\big) \nabla_\theta \log \pi_\theta(a_t \mid h_t) \right],$$

只要 $b(h_t)$ 只依赖到第 t 步为止的历史（不依赖未来动作），它就是无偏的。证明的核心是：策略的总概率对参数求导的期望为零，

$$\mathbb{E}_{a\sim\pi_\theta}\left[b(h)\nabla_\theta \log \pi_\theta(a \mid h)\right] = b(h)\sum_a \pi_\theta(a \mid h)\nabla_\theta \log \pi_\theta(a \mid h) = b(h)\sum_a \nabla_\theta \pi_\theta(a \mid h) = b(h)\nabla_\theta 1 = 0,$$

其中第一个等号把期望展开成对动作求和，第二个等号用 $\pi\nabla\log\pi = \nabla\pi$，最后一步用「概率之和恒为 1」。所以减去 baseline 只改方差、不改期望。

baseline 的作用

baseline 让「表现比预期好」的动作得到正优势、比预期差的得到负优势；它不改变梯度的期望，只降低方差。RL 工程里大量工作都在寻找更好的 advantage/基线表示——这决定了训练是否稳定。

6.4 二动作例子：方差如何被压低

课堂用一个只有两个动作的 bandit 例子把「降方差」算清楚：策略只有一个参数 θ，用 sigmoid 把它压到 (0,1) 得到选 A 的概率。取 $\theta = -1.1$ 时，$p(A) = 0.25$（奖励 1）、$p(B) = 0.75$（奖励 0）——这是一个坏策略（把更大概率放在无奖励动作上），训练应该把 θ 往正方向推。

精确的策略梯度为 0.18，但 REINFORCE 的采样估计只有两种：采样到 A 时得到 0.75，采样到 B（奖励 0）时得到 0——噪声很大。若减去基线 $b = 0.5$，两条样本的估计变成 0.375 与 0.125，期望仍是 0.18，但方差小得多，梯度更新更稳定。

课堂还展示了重复采样的可视化：不减去 baseline 时估计值在 0 与 0.75 之间大幅跳动，收敛很慢；减去 baseline 后估计更贴近真实值，收敛更快。基线可以是常数（如 0.5）、历史奖励的滑动平均，也可以是一个训练出来的价值模型（actor-critic / PPO）。

中间奖励与 reward-to-go

如果环境在中间步骤给奖励，实务上把「奖励」换成「未来奖励之和」（reward-to-go）$R_t = \sum_{t'=t}^{T} r_{t'}$，并只让动作对之后的奖励负责。这既符合因果，也能进一步降低方差；Number Search 只在结束时给奖励，所以 R_t 对所有动作相同。

6.5 本章小结

REINFORCE 对失败轨迹没有信号，因此奖励应当是相对的：减去一个只依赖历史的 baseline 得到优势，既无偏又降方差。二动作 bandit 例子把「真实梯度 0.18、估计 0.75/0、减基线后 0.375/0.125」算给读者看——这是理解 advantage 为什么重要的最短路径。

7 GRPO 与 Dr. GRPO：用组均值当基线

7.1 GRPO 的核心：同题多采样，组均值作 baseline

GRPO（group-relative policy optimization）的想法非常直接：既然好的 baseline 就是「该题上的期望奖励」，那就**对同一道题采样多条轨迹**，用组内奖励均值当 baseline。这样不需要额外的价值模型（PPO 的 critic），省下大量显存与算力——这也正是 DeepSeek 提出它时的效率动机。

具体做法：对每个任务实例 x_i 采样 G 条轨迹（常见 $G \approx 8$），计算奖励并求组均值与标准差，得到每条轨迹的优势：

$$\hat{A}_j = \frac{R(\tau_j) - \bar{R}}{\sigma_R}, \qquad \bar{R} = \frac{1}{G}\sum_{j=1}^{G} R(\tau_j),$$

其中 $R(\tau_j)$ 是第 j 条轨迹的奖励，$\bar{R}$ 是组均值（baseline），σ_R 是组内奖励标准差。课堂用一组 4 条轨迹的例子演示：奖励为 $[1,1,0,0]$ 时 $\bar{R}=0.50$、$\sigma_R=0.5$，优势为 $[+1,+1,-1,-1]$；换一个更难的实例（奖励 $[1,0,0,0]$，$\bar{R}=0.25$），优势变成 $[+1.73,-0.58,-0.58,-0.58]$；更简单的实例（奖励 $[1,1,1,0]$，$\bar{R}=0.75$）则为 $[+0.58,+0.58,+0.58,-1.73]$。

对应的损失把 REINFORCE 的「每条轨迹一个奖励」换成「每条轨迹一个组内优势」：

$$\mathcal{L}_{\text{GRPO}} = -\sum_{i}\frac{1}{G}\sum_{j=1}^{G}\hat{A}_{ij}\sum_{t}\log \pi_\theta(a_{ij,t} \mid h_{ij,t}),$$

其中 i 是 batch 内的任务实例，j 是组内第 j 条 rollout，t 是轨迹内的时间步。注意组均值和归一化只在**同一实例内**做——优势必须相对同一道题才有意义。

为什么 GRPO 的 baseline 无偏

baseline $b(h_t) = \bar{R}$ 只依赖 prompt 和模型自己采样的结果，不依赖当前参数对某个动作的预测；按第 6 节的无偏性论证，减去它不改变梯度期望。原论文还乘了一个重要性比率并做 clipping、加 KL 正则来稳定训练，这些留到下一讲。

7.2 Dr. GRPO：两处修正

课堂特别介绍了 Dr. GRPO（“GRPO done right”）的两处改动，并说明作业里也会采用：

	GRPO	Dr. GRPO	为什么改
优势	$\frac{R_j - \bar{R}}{\sigma_R}$	$R_j - \bar{R}$	除以标准差会把不同难度的题「标准化」到同一尺度，优势不再感知题目难度
响应归一化	$\frac{1}{\sum_t 1}\sum_t(\cdot)$	$\frac{1}{C}\sum_t(\cdot)$	按实际生成长度归一化会偏向冗长的错误答案；C 是固定的生成长度预算（常数）

其中 C 是「这条回复最多能生成多少 token」的固定常数，对每条回复相同，所以奖励保持固定、响应项的相对权重不再随长度漂移。

7.3 全零/全一奖励：没有组内方差就没有信号

GRPO 有一个结构性弱点：如果组内所有轨迹**全部失败**（奖励全 0），组均值也是 0、优势全为 0，没有梯度；反之如果**全部成功**（奖励全 1），同样没有梯度。前者说明题太难，后者说明题太简单。课堂给了几条应对：换题目、往 prompt 里注入提示（引导模型产出可能正确的解）、加入示范、把正确示范放进组里（off-policy RL）、或先用示范做 SFT；也可以设计更有信息量的中间奖励（reward shaping，例如奖励「找到了正确文件」）。

组内无方差

GRPO 的采样算力花在「组内有对有错」的题上才有效。工程上要监控每批的「全对组/全错组比例」，并按难度动态筛题——这是 RL 训练里最常被忽视的效率坑之一。

7.4 GRPO 与 PPO 的关系

PPO 的做法是训练一个与策略共享主干的价值模型来预测 baseline；GRPO 则用「同一问题的多条 rollout 的组均值」替代这个价值模型，省掉了一个模型和相应的显存。课堂用一张图对比了两者的结构：PPO 有 value model 与优势估计，GRPO 直接对组内奖励做归一化得到优势。

7.5 本章小结

GRPO = REINFORCE + 「同题多采样、组均值作 baseline」，在组内归一化得到优势，省掉 critic；Dr. GRPO 去掉按标准差缩放、改用固定长度常数归一化，避免难度感知丢失与长度偏置。组内全对/全错没有信号，需要课程、提示、示范或中间奖励来救。

8 三种方法的总账：都在「重加权生成的动作」

课堂最后把 REST、REINFORCE、GRPO 放在一起比较：三者都让 agent 在环境里行动、都用生成的动作算损失（都像 self-training），差别只在**如何重加权**：

方法	加权方式	适用/代价
REST	二值筛掉失败轨迹，只在成功轨迹上做 SFT	简单、复用 SFT 工程；依赖能采到成功轨迹
REINFORCE	用轨迹奖励 $R(\tau)$ 缩放对数概率	通用、无偏；方差大、失败轨迹无梯度
GRPO	用组内相对优势 $\hat{A}_j$ 缩放，同题多采样	降方差、省掉 critic；组内无方差时无信号

三者都解决了开头的三个 SFT 问题：**任务失配**（直接最大化期望奖励）、**数据失配**（失败轨迹也能通过相对优势提供负信号）、**暴露偏差**（轨迹由当前策略生成，模型在训练中就练习从自己的错误里恢复）。代价是必须真的在环境里探索，采样成本高、梯度有噪声——这正是后两讲（高级算法与 RL 系统）要继续处理的问题。

9 总结与延伸

本讲从「SFT 的不足」出发，把 RL 的核心公式一路推到 GRPO。可以压缩成六条：

1. SFT 有三个问题：任务失配（模仿动作 vs 最大化奖励）、数据失配（浪费失败轨迹）、暴露偏差（训练历史与推理历史不一致）。
2. RL 形式化：轨迹概率 = 策略项 × 环境项；目标是 $J(\theta) = \mathbb{E}[R(\tau)]$，环境未知也能靠采样估计。
3. REST 是最简桥梁：采样 → 按奖励筛选 → SFT → 循环；好轨迹不够时用冷启动/课程/中间奖励。
4. REINFORCE 用对数导数技巧得到 $\nabla J = \mathbb{E}[R(\tau) \sum_t \nabla \log \pi_\theta]$，实现上就是「mask + 奖励缩放的 SFT」。
5. 奖励应当相对：减去只依赖历史的 baseline 不改变梯度期望、只降低方差；优势 $A = Q - V$ 让「比预期好/差」的动作分别被强化/抑制。
6. GRPO 用同题多采样的组均值当 baseline（省掉 critic），在组内归一化得优势；Dr. GRPO 去掉按标准差缩放、把响应归一化改成固定常数，避免难度感知丢失与长错误答案偏置。组内全对/全错没有信号。

一句话记住本讲

策略梯度 = 用奖励给「模型自己生成的轨迹」加权；baseline 让奖励变成「比预期好多少」；GRPO 用同题多采样的组均值当那个「预期」，从而在不训练价值模型的情况下降低方差。

延伸思考：(1) 把 REINFORCE 的损失与 SFT 的损失并排看，会发现二者的唯一区别是「要不要乘奖励、乘什么样的奖励」——这解释了为什么语言模型 RL 的实现看起来简单、难的是采样系统与稳定性；(2) 从「相对奖励」的角度看，RLVR 的二值奖励本身信息量很低，真正提供信号的是组内的相对比较，所以「组大小、题目难度分布、全对全错比例」这些采样设计，可能比损失函数细节更影响最终效果；(3) 本讲刻意省略了重要性比率、clipping 与 KL 正则——它们是训练稳定的关键，也是下一讲（高级 RL 算法）的主线。下一讲将讨论 off-policy 方法、稳定化技巧与 RL 系统实践。